



Università degli Studi di Messina

System Security Course

End-to-End Encrypted

Chat Application

CipherWhisper

Student: Batoul Alhaj Ali Hamieh

Matricola: 550149

Table Of Contents

1 .INTRODUCTION

2 SYSTEM ARCHITECTURE

3.END -to- END Encryption (E2EE) Design

4. FUNCTIONALITIES

5 .DATABASE DESIGN

6 .SECURITY MEASSURES

7 . CONCLUSION

8 . REFERENCES

1. Introduction

CipherWhisper is a secure chat application developed with the primary goal of protecting user communication through **End-to-End Encryption (E2EE)**. In traditional chat applications, the server can potentially access message content, which poses serious privacy and security risks.

The main objective of this project was to design a system where the server remains completely untrusted. All encryption and decryption operations are performed on the client side, ensuring that even if the server is compromised, user messages remain protected.

Key security features implemented in CipherWhisper include:

- Client-side RSA key generation
- Dual encryption mechanism
- Brute-force protection on login (account lockout after five failed attempts)
- Secure password storage and session management

By shifting all cryptographic operations to the client and minimizing trust in the server, CipherWhisper aims to provide strong confidentiality and privacy for its users.



2. System architecture

CipherWhisper follows a client-server architecture designed with a strong security focus. The main goal was to minimize trust in the server by performing all cryptographic operations on the client side.

The system consists of three main components:

Client Side

The frontend is built using HTML, CSS, and JavaScript. All encryption and decryption are handled directly in the browser using the Web Cryptography API. During registration, each user generates an RSA key pair on their device. The public key is sent to the server, while the private key is downloaded locally as a .pem file and never leaves the user's device. This design ensures that even if the server is compromised, user private keys remain secure.

Server Side

The backend is developed using PHP running on an Apache server. The server's role is limited to user authentication, storing public keys, and delivering encrypted messages. The server never performs encryption or decryption and has no access to plaintext messages or private keys. This significantly reduces the attack surface on the server.

Database

The detailed structure of the users and messages tables is described in the Database Design section. Messages are stored only in encrypted form, no plaintext messages or private keys are ever saved in the database.

The communication flow follows a secure model:

Client (Encryption) → Server (Storage & Delivery) → Client (Decryption)

To simplify deployment and ensure consistency across different environments, the application was containerized using Docker. This ensures that the environment is isolated and reproducible.

3. End-to-End Encryption (E2EE) Design

CipherWhisper implements true End-to-End Encryption by performing all cryptographic operations on the client side. The server is treated as an untrusted entity that only stores and delivers encrypted data. This section describes the encryption workflow used in the system.

3.1 RSA Key Pair Generation (Client-Side)

Generating the RSA key pair on the client side ensures that the private key never leaves the user's device.

```
const keyPair = await window.crypto.subtle.generateKey({
  name: "RSA-OAEP",
  modulusLength: 2048,
  publicExponent: new Uint8Array([1, 0, 1]),
  hash: "SHA-256",
}, true, ["encrypt", "decrypt"]);

const publicKey = await window.crypto.subtle.exportKey("spki", keyPair.publicKey);
const publicKeyBase64 = btoa(String.fromCharCode(...new Uint8Array(publicKey)));
```

Along with the public key, the user's email and hashed password are sent to the server during registration.

Passwords are hashed before being stored in the database.

```
const response = await fetch('register.php', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    email: email,
    password: password,
    public_key: publicKeyBase64
  })
});
```

The server only stores the public key and the hashed password.

```
$hashed_password = password_hash($password, PASSWORD_DEFAULT);

$sql = "INSERT INTO users (email, password, public_key) VALUES ('$email', '$hashed_password', '$public_key')";
```

3.2 Fetching the Recipient's Public Key

Before sending a message, the client retrieves the recipient's public key from the server using the `get_public_key.php` endpoint.

Client Side (JavaScript)

The following function is used in `chat.js` to fetch and import the recipient's public key

```
async function getPublicKey(userId) {
  const data = await (await fetch(`get_public_key.php?receiver_id=${userId}`)).json();
  return window.crypto.subtle.importKey(
    'spki',
    Uint8Array.from(atob(data.public_key), c => c.charCodeAt(0)).buffer,
    { name: 'RSA-OAEP', hash: 'SHA-256' },
    false,
    ['encrypt']
  );
}
```

Server Side (PHP)

When the client requests a user's public key, the server uses the following code in `get_public_key.php`:

```
$receiver_id = intval($_GET['receiver_id']);

$sql = "SELECT public_key FROM users WHERE id = ?";
$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $receiver_id);
$stmt->execute();
$result = $stmt->get_result();
```

This ensures that only the recipient's public key is returned to the client. No private keys or other sensitive data are exposed.

3.3 Dual Encryption Mechanism

To maintain true end-to-end encryption while allowing both the sender and the recipient to read messages, CipherWhisper uses a dual encryption approach on the client side.

The Problem: If a message was encrypted only with the recipient's public key, the sender would not be able to read their own sent messages later, because they do not have the recipient's private key.

The Solution: Every message is encrypted **twice** on the client side before being sent to the server:

- First, the message is encrypted using the **recipient's public key**. This ciphertext is stored in the message column of the database. Only the recipient can decrypt it using their private key.
- Second, the same message is encrypted using the **sender's public key**. This ciphertext is stored in the sender_encrypted_message column. Only the sender can decrypt it using their own private key.

This way, both the sender and the recipient can read the message using their respective private keys, while the server never sees any plaintext.

When a user opens a chat and uploads their private key, the system checks whether the current user is the sender or the recipient of each message. It then selects the correct ciphertext (message or sender_encrypted_message) and decrypts it locally in the browser using the Web Cryptography API. This ensures that end-to-end encryption is preserved while still allowing users to view their own sent messages.

```
const [receiverKey, senderKey] = await Promise.all([getPublicKey(receiverId), getPublicKey(senderId)]);
const encodedMessage = new TextEncoder().encode(message);

await fetch('store_message.php', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    receiver_id: receiverId,
    message: btoa(String.fromCharCode(...new Uint8Array(await window.crypto.subtle.encrypt({ name: 'RSA-OAEP' }, receiverKey, encodedMessage)))),
    sender_encrypted_message: btoa(String.fromCharCode(...new Uint8Array(await window.crypto.subtle.encrypt({ name: 'RSA-OAEP' }, senderKey, encodedMessage))))
  })
});
```

3.4 Storing Encrypted Messages

The server stores only the encrypted messages in the database. It never receives or stores plaintext messages or private keys. The following code is used to insert the encrypted messages:

```

$sql = "INSERT INTO messages (sender_id, recipient_id, message, sender_encrypted_message) VALUES (?, ?, ?, ?)";
$stmt = $conn->prepare($sql);
$stmt->bind_param("iiss", $sender_id, $receiver_id, $message, $sender_encrypted_message);

if ($stmt->execute()) {
    echo json_encode(['success' => true]);
} else {
    echo json_encode(['error' => $stmt->error]);
}

```

Showing rows 0 - 13 (14 total, Query took 0.0007 seconds.)

SELECT * FROM `messages`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	Id	sender_id	recipient_id	message	sender_encrypted_message
☐ Edit ☐ Copy ☐ Delete	1	8		a5HQmcXZyHgkQlaA5+AGMMk3D0zMX8OT3Bchi1mPz9LrIYepuG...	G09jSqNoTczOpl6x5lm/rO56++1W2lCcVlwoI5ukRzuJ023wQt...
☐ Edit ☐ Copy ☐ Delete	2	1		8 pjcOQJzQV45B7U8lUhlw1nOcvpKw+8nAqbtVy55fm/65z3MF...	V6ehseBtoFtrmwicmZ85lQZweKmbmHwYQJo+1Qf/Ue41xiWFGl...
☐ Edit ☐ Copy ☐ Delete	3	11		1 AahrhGGN2nnAbblU+VWmWbBKB+eefzDa67dg4wYjMkusZKVPd2...	V9GGFdOEgottoMhsOf7xRieFj37bsH0lrv7KVT+7WhclZDTw...
☐ Edit ☐ Copy ☐ Delete	4	11		1 hkOMLcVn0VcTn+IRYx8OBbQlKaLJgnU26v3ku1ibeSL/QVo4o...	Easl92VBW4XOMHROw6ut3GqM7bsluHHlIWZO8UHZGzIzelyHa...
☐ Edit ☐ Copy ☐ Delete	5	1		11 HO3NHv4gat3xdAw+uW44XK++AGJWLXaYqDUBlrb3MOlbZTEpp...	p2qivhu1LSdRIA4D0rZ9hMP2mQwAuWlVv8vLMEL1PoBplGwRN...
☐ Edit ☐ Copy ☐ Delete	6	1		11 faqFgae7nK4RO+/EX5sh1wjs9A1Ochum8WtmkVqlyNbOxINByQ...	StLWdWYp7hcVIVXolCwI1+ZasqfWQ64ehMgewQYmiA7AmssYU...
☐ Edit ☐ Copy ☐ Delete	7	14		6 pxUJnr2oFF6Z07QzUJRh+sFqVR2evKEgGdP9bsBg6zp5buVRT9...	E8NgSY/DKAFXWxJlRlROFcZmROn2HYtCvUbgLnJluVYAFvmgp...
☐ Edit ☐ Copy ☐ Delete	8	6		14 J9VX3Q3JYM0wREYhsY+XYOC9v7+MZIW1AVH+E++vRCO11jaq9...	dQ4YP8kiYrkGm3p3rYu8k28gxyBG9YEm1w82UJKHKNvE0Sq0Z...
☐ Edit ☐ Copy ☐ Delete	9	14		1 DmXDeu2DkcyGDmazQlIHfMVva4FrEpMwBqznLAJ9+mVu5zcy...	izzVZPobX8+bDgDoo2lm5WOQOCXlyS2eLr/inqu/eS1gXgalZ...
☐ Edit ☐ Copy ☐ Delete	10	14		2 XC6k9KaYIOQ8F56mucG8R6aYX6mTymOkJluPFydx2te9+XuRyb...	PoCruluTSB8qE1y+go400aEjGhthvNsgPGSX7jWLRUHH2CK30...
☐ Edit ☐ Copy ☐ Delete	11	14		10 IF75zzOytnbV46QTxhZ9hboYA/nWcoMErWw9OPW7nUEZ9HXvf...	K4ZyZ4kF4E8Rpkppljx4K7GLInjetQBButUXOmXlRlGRRhP8...
☐ Edit ☐ Copy ☐ Delete	12	14		12 vIHkIqYiNePGSdSbby2OW1bl18pB08AeD1G/6XiYXljw5exOD...	hxSDnc3+Yy79Poc0smvAhsCncbVZsDVkt3wKA9r6ArUfalY12q...

3.5 Decrypting Messages on the Client Side

When a user opens a chat, they must upload their private key. Messages are then decrypted locally in the browser.

Once the key is loaded, the encrypted messages are retrieved from the server. Depending on whether the current user is the sender or the recipient, the appropriate ciphertext (message or sender_encrypted_message) is selected and decrypted locally in the browser using the user's private key. The decrypted plaintext messages are then displayed in the chat interface.

All decryption operations are performed entirely on the client side

```

async function decryptMessage(privateKey, encryptedMessage) {
    return new TextDecoder().decode(await window.crypto.subtle.decrypt(
        { name: 'RSA-OAEP' },
        privateKey,
        Uint8Array.from(atob(encryptedMessage.trim()), c => c.charCodeAt(0))
    ));
}

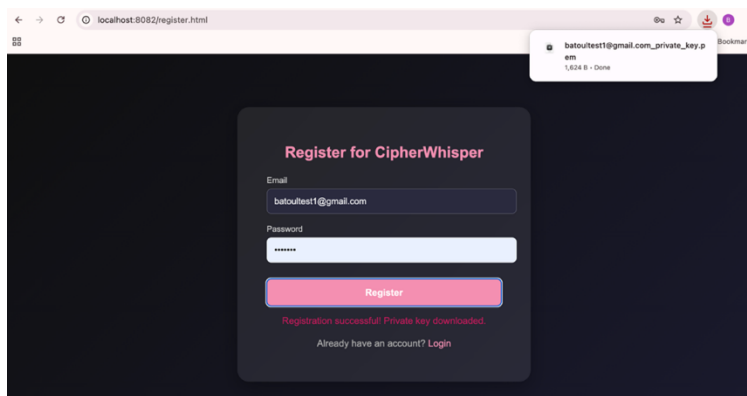
```

4. Functionality

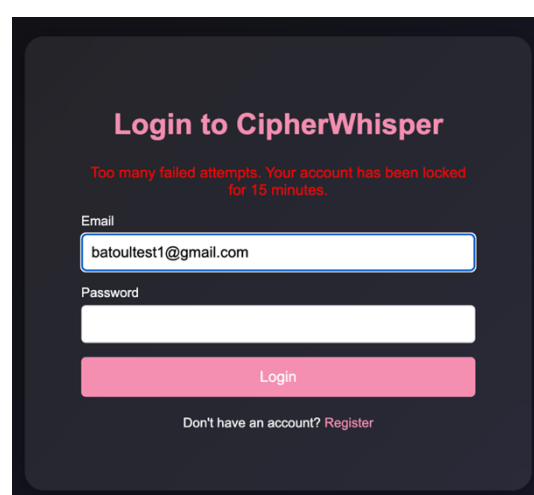
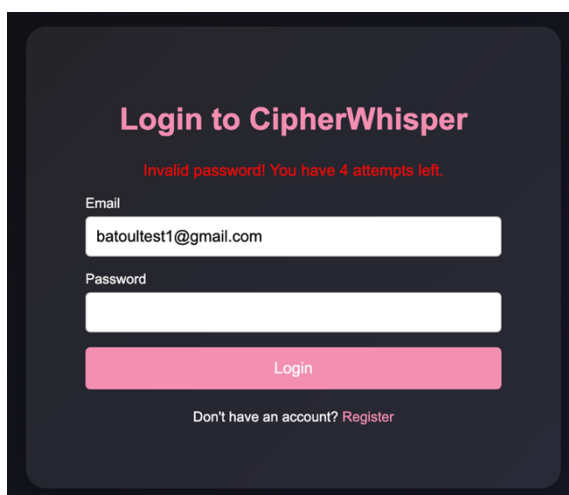
CipherWhisper provides the following core functionalities:

4.1 User Registration and Key Generation

During registration, each user generates an RSA key pair directly in the browser. The public key is sent to the server, while the private key is automatically downloaded as a .pem file. This ensures that private keys never leave the user's device.



4.2 Secure Login with Brute-Force Protection



The login system in CipherWhisper includes protection against brute-force attacks. If a user enters an incorrect password five consecutive times, their account is automatically locked for 15 minutes. This security feature significantly reduces the risk of unauthorized access attempts.

To implement this protection, the system tracks failed login attempts in the database and locks the account when the limit is reached.

Checking if the account is locked:

```
if (!empty($user['locked_until']) && strtotime($user['locked_until']) > time()) {  
    $error = "Your account is locked due to too many failed attempts. Please try again later.";
```

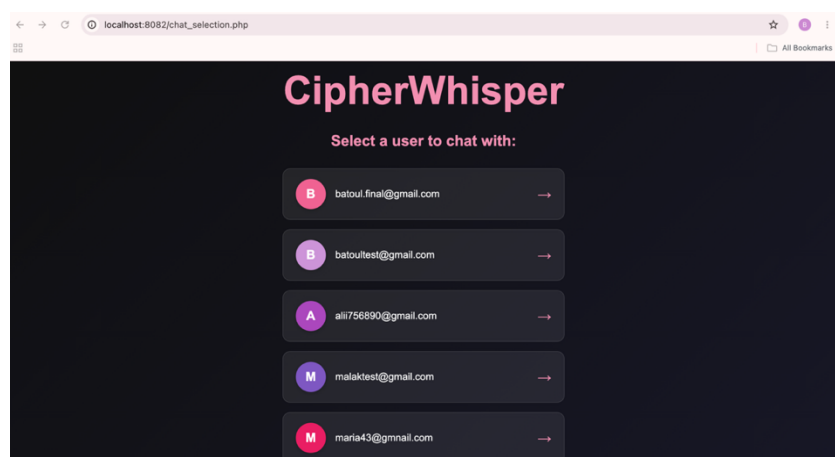
Handling failed attempts and account lock:

```
$new_attempts = $user['failed_attempts'] + 1;  
$update_sql = "UPDATE users SET failed_attempts = $new_attempts, last_failed_attempt = NOW() WHERE id = " . $user['id'];  
mysqli_query($conn, $update_sql);  
  
if ($new_attempts >= 5) {  
    $lock_sql = "UPDATE users SET locked_until = DATE_ADD(NOW(), INTERVAL 15 MINUTE) WHERE id = " . $user['id'];  
    mysqli_query($conn, $lock_sql);  
    $error = "Too many failed attempts. Your account has been locked for 15 minutes.";  
} else {  
    $remaining = 5 - $new_attempts;  
    $error = "Invalid password! You have $remaining attempts left.";  
}
```

After a successful login, the failed attempt counter is reset to zero, allowing the user to log in normally again.

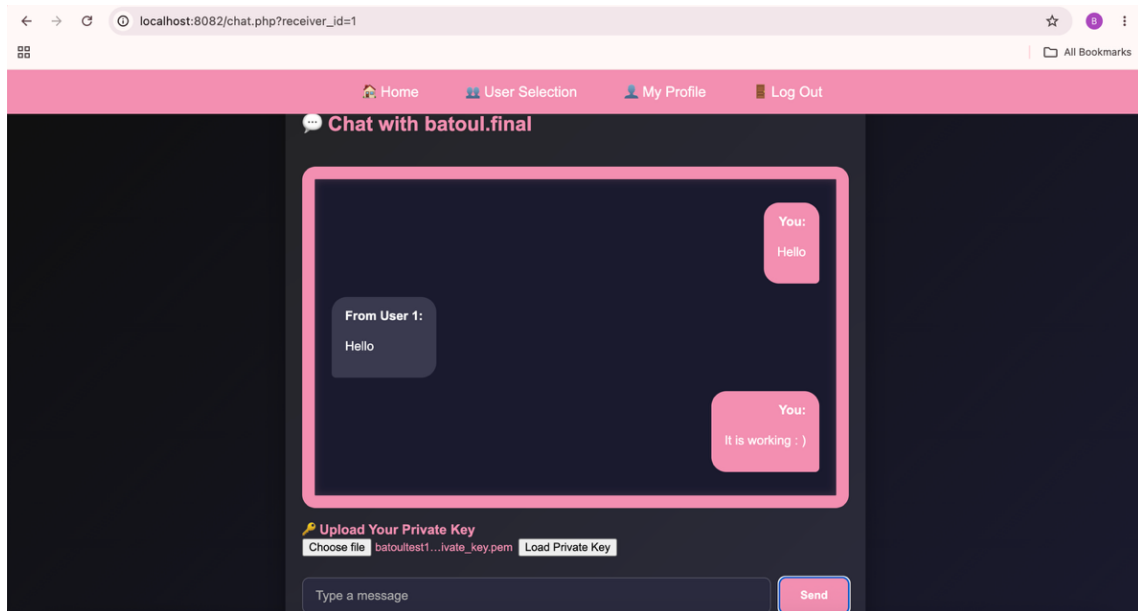
4.3 User Selection

After logging in, users can view a list of registered users and select the person they want to chat with. Each user is displayed with a colored avatar and their email address.



4.4 Secure Chat Interface

The chat interface allows users to send and receive messages securely. Users must upload their private key to decrypt messages. All cryptographic operations are handled as described in Section 3. New messages are fetched periodically from the server using AJAX polling.



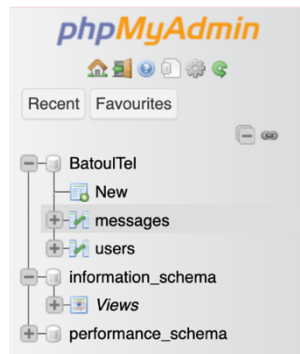
4.5 Profile Management

Users can view their email and registration date on the profile page. A direct “Start Chatting” button allows them to quickly begin a conversation.



5. Database Design

CipherWhisper uses a MySQL database to store user information and encrypted messages. The database was set up using Docker and phpMyAdmin for easy management.



The database consists of two main tables:

1. users Table

This table stores all registered users along with their public keys and login security information.

Columns:

- **id** : Unique identifier for each user
- **email** : User's email address (Unique)
- **password** : Hashed password
- **public_key** : User's RSA public key (used for encryption)
- **created_at** : Timestamp of registration
- **failed_attempts** : Number of consecutive failed login attempts
- **last_failed_attempt** : Timestamp of the last failed login
- **locked_until** : Timestamp until which the account is locked (after 5 failed attempts)

A screenshot of the phpMyAdmin 'Table structure' view for the 'users' table. The table has 8 columns: id, email, password, public_key, created_at, failed_attempts, last_failed_attempt, and locked_until. Each column has a checkbox, a name with a key icon, a type, a collation, attributes, null status, default value, comments, extra options, and an action menu (Change, Drop, More).

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	email	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
☐ 3	password	varchar(255)	utf8mb4_0900_ai_ci		No	None			Change Drop More
☐ 4	public_key	longtext	utf8mb4_0900_ai_ci		Yes	NULL			Change Drop More
☐ 5	created_at	timestamp			Yes	CURRENT_TIMESTAMP		DEFAULT_GENERATED	Change Drop More
☐ 6	failed_attempts	int			No	0			Change Drop More
☐ 7	last_failed_attempt	datetime			Yes	NULL			Change Drop More
☐ 8	locked_until	datetime			Yes	NULL			Change Drop More

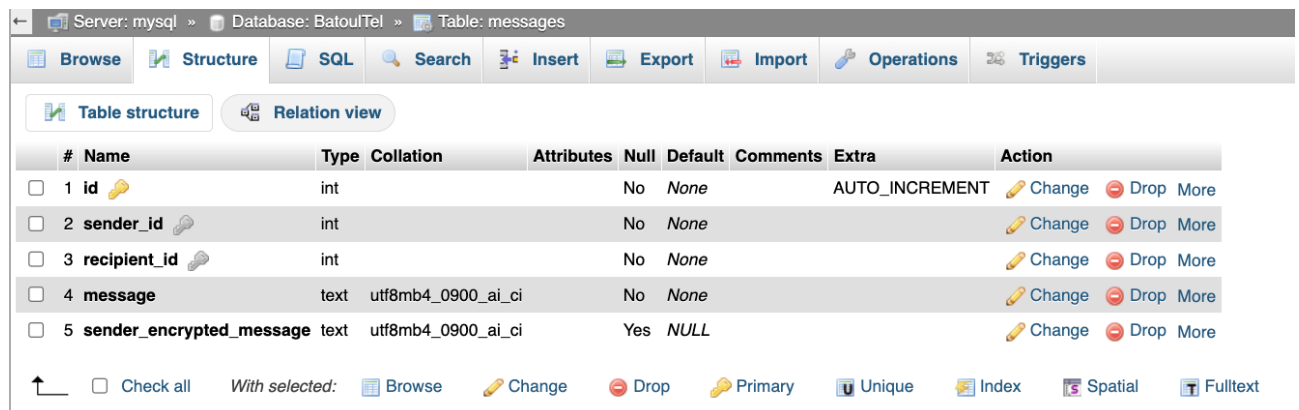
2. messages Table

This table stores all encrypted messages exchanged between users.

Columns:

- `id` : Unique message identifier
- `sender_id` : ID of the user who sent the message
- `recipient_id` : ID of the user who receives the message
- `message` : Message encrypted with the recipient's public key
- `sender_encrypted_message` : Message encrypted with the sender's public key (allows sender to read their own messages)

Foreign key constraints are applied between `messages.sender_id` → `users.id` and `messages.recipient_id` → `users.id` to maintain data integrity.



The screenshot shows a MySQL database management interface with the following table structure:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id 🔑	int			No	None		AUTO_INCREMENT	Change Drop More
2	sender_id 🔑	int			No	None			Change Drop More
3	recipient_id 🔑	int			No	None			Change Drop More
4	message	text	utf8mb4_0900_ai_ci		No	None			Change Drop More
5	sender_encrypted_message	text	utf8mb4_0900_ai_ci		Yes	NULL			Change Drop More

Below the table structure, there are several icons and labels for table operations: ☐ Check all With selected: Browse Change Drop Primary Unique Index Spatial Fulltext

6. Security Measures

CipherWhisper implements multiple layers of security to protect user data and communications:

1. End-to-End Encryption (E2EE)

The server only stores encrypted data and never performs any cryptographic operations. (Section 3)

2.Client-Side Cryptography

All cryptographic operations are performed on the client side ,the server never sees the original content of the messages keeping all conversations fully private. (Section 3)

3.Secure Password Storage

User passwords are never stored in plaintext. They are securely hashed using PHP's password_hash() function before being saved in the database.

4.Brute-Force Attack Protection

The brute-force protection mechanism (account lockout after five failed attempts) was implemented to mitigate online password guessing attacks, which are a common threat against authentication systems

5.SQL Injection Prevention

To protect the database from malicious attacks prepared statements and proper query validation is used. This prevents hackers from injecting harmful SQL code into the databse.

6.Secure Session Management

Users need to log in securely with their email and password. Sessions are managed ensuring only authenticated users can access their accounts and data.

Limitations

While CipherWhisper successfully implements End-to-End Encryption, the project has some limitations. Due to the use of direct RSA encryption, message length is restricted. Additionally, the system does not provide forward secrecy, meaning that if a user's private key is compromised in the future, previously sent messages could be decrypted. Users must also manually upload their private key file each time they want to read messages, and losing this file results in permanent loss of access to historical messages.

7. Conclusion

In this project, I developed **CipherWhisper**, a secure chat application focused on End-to-End Encryption. All encryption and decryption operations are performed on the client side using the Web Cryptography API, ensuring that private keys never leave the user's device.

A key feature of the system is dual encryption, where every message is encrypted twice — once for the recipient and once for the sender. This maintains true end-to-end encryption while allowing users to view their own messages.

Additional security measures were implemented, including brute-force protection), secure password hashing, and SQL injection prevention. The application was also containerized using Docker for easier deployment.

Overall, CipherWhisper successfully demonstrates a practical implementation of End-to-End Encryption in a web-based chat application, providing both security and usability.

REFERENCES

1. [Web Cryptography API Documentation](#)
2. [PHP Official Documentation](#)
3. [MySQL Documentation](#)
4. [RSA Encryption Algorithm](#)
5. [JavaScript Fetch API](#)
6. [Best Practices for End-to-End Encryption \(E2EE\)](#)
7. [Stack Overflow Discussions](#)
8. [Official PHP PDO Documentation](#)